Codex Blue Book: From Project Rules to Verified Delivery

v1.4.0 · Sources checked October 21, 2026



Author: Hunk Wu (X: @ai_pmer)

中文 PDF | 中文在线阅读

Table of Contents

Part 1: Getting Started

- Ch.01 From Code Generation to Verified Results
- Ch.02 Install Codex and Check Your Environment
- Ch.03 Diagnose Sandbox Access to Local Services

Part 2: Tasks and Project Rules

- Ch.04 Guide Agents with Goals and Acceptance Checks
- Ch.05 Record Project Rules in AGENTS.md
- Ch.06 Inspect Progress and Correct Course

Part 3: UI and Mobile Workflows

- Ch.07 Validate Frontends in a Real UI
- Ch.08 Route Build Alerts and Approvals to Mobile

- Ch.09 Baseline and Refactor Legacy Code Incrementally

Part 4: Refactoring and Delivery

- Ch.10 Validate a Subscription SaaS Payment Flow
- Ch.11 Build and Check a Mobile App with Expo
- Ch.12 From Product Usage to Acquisition Experiments

Part 5: Growth and Version Changes

- Ch.13 Check 2026 Model and Tool Changes

Ch.01 From Code Generation to Verified Results

> ***Problem**: Treating AI as autocomplete or accepting edits without review can miss requirements and regressions.*
>
> ***Practice**: Write one development task with a goal, boundaries, and verifiable acceptance checks.*

When chatting with readers of "实战产品说" (Real-World Product Talk), I often notice a common pitfall: developers pushing themselves to memorize AI coding commands and shortcuts as if they were cramming for an API manual.

Coding agents such as Codex can help draft boilerplate, tests, and documentation. Developers still own requirements, key design decisions, and verification. A clear goal, edit scope, and acceptance checks matter more than memorizing prompts.

This chapter turns a product need into a development task that can be executed and checked.

A Way to Think About It: Stop Being a Bricklayer, Become the Chief Architect

If you are new to AI-native development, do not view AI as a cold code generator:

```
【Traditional Handwritten Code】 ——> You act as a manual bricklayer, stacking bricks one by one.
If one corner is misaligned, the whole building collapses.
【Traditional AI Completion】    ——> You lay bricks while AI hands you mortar (you type a line,
it guesses half; your hands never rest).
【Codex Orchestration Era】      ——> You are the "Chief Construction Architect", and Codex is your
"elite engineering crew":
                                    - You sketch blueprints (Define the Goal);
                                    - You erect safety fences (Enforce Sandboxes & Constraints);
                                    - You inspect structural safety with laser levels (Automated
Validation);
                                    - The crew (Codex) calculates structural mechanics, procures
concrete, and builds the walls.
```

When your focus changes from "how do I type this loop" to "how do I dispatch a foolproof work order to my crew", you have officially stepped into AI-native engineering.

1.1 The Evolution of AI Coding: Where Are We?

```
+------------------------------------------------------------+
| Stage 1: Assisted Completion (Copilot)                     |
```

```
| - Experience: Input suggestions. You type, it guesses.       |
+--------------------------------------------------------------+
                               |
                               ▼
+--------------------------------------------------------------+
| Stage 2: Terminal Dialog & Execution (Claude Code / Cursor) |
| - Experience: Outsourced developer. You prompt, it edits.    |
+--------------------------------------------------------------+
                               |
                               ▼
+--------------------------------------------------------------+
| Stage 3: Autonomous Multi-Surface Agent (OpenAI Codex)       |
| - Experience: Digital full-stack team. Sandbox testing,      |
|   multi-surface (IDE/CLI/Mobile/Desktop) orchestration.      |
+--------------------------------------------------------------+
```

Codex has pushed us into Stage 3:

- **Multi-Surface Productivity Matrix**: Automated execution via CLI, complex reviews and Sites in-place preview on ChatGPT Desktop (Code Mode), 24/7 mobile monitoring with Guardian approvals, and visual browser audits via Desktop Computer Use.
- **Frontier Reasoning & Agentic Core**: From **GPT-6 Astra** for the hardest cross-tool work to **GPT-6.1 Sol** for everyday complex coding and **GPT-6 Luna** for focused tasks, choose a model by task and verify tools, permissions, and results in your own environment. Current model guidance

---

1.2 The Core Shift: From Process Control to Boundary Control

As a Product Manager (PM), you know that when writing a PRD, you don't instruct developers on "how to structure their loops." You define acceptance criteria and boundary constraints.

With Codex, the same rule applies. Stop micromanaging how AI writes logic. As an orchestrator, focus on three things:

1. **Define the End State (Goal)**: What problem are we solving for users?
2. **Establish Guardrails (Constraints)**: Which modules are off-limits? How are secrets isolated?
3. **Automate Validation (Validation)**: Write test suites so Codex can prove its work is correct.

> *[!IMPORTANT]* ***Orchestrator Formula***
> `Successful Delivery = Clear Goal + Strict Constraints + Automated Testing`

Practice: Start with Three Steps

For first-time users, avoid dumping thousands of words of vague thoughts. Follow this 3-step loop:

1. **Step 1: Draft a 50-Word Requirement Card**
   Clarify 3 elements: Target page/feature (Goal), existing framework/libraries (Constraints), validation command (Validation).
2. **Step 2: Restrict Codex in a Safe Sandbox**
   Execute in the terminal with workspace write permission: `codex exec --sandbox workspace-write`.
3. **Step 3: Require the Agent to Prove Verification**
   Instruct Codex: "Run `npm run build` after editing and ensure exit code 0 before completing".

🛡️ Troubleshooting & Pitfall Cheat Sheet

Symptom	Root Cause	Instant Fix

Treating AI as a search engine for theory	Hours of conversation without a single line of working code	Stop chatting; dispatch a concrete spec: "Create a login card in `src/` and pass tests"
Pasting 10,000 lines of unrelated code	Context bloated with noise, causing hallucinations	Point out 1-2 specific files to keep agent edits surgically targeted
Trusting AI claims without running tests	AI assumed it fixed the bug, but runtime crashes	Add mandatory validation specs: "Must run `npm test` in sandbox and print green pass report"

1.3 Your New Moat: Orchestration and Domain Insight

When code is commoditized, what is your moat?

As shared on pmer.cn: **Your value lies in defining business boundaries and empathizing with user pain points.** You no longer need to spend three days tweaking CSS flexbox centering or wrestling with Webpack errors. Your core job is to structure architecture with Codex, deploy fast, and let the market validate your commercial MVP.

Ch.02 Install Codex and Check Your Environment

> ***Problem**: CLI, dependencies, or permissions can fail before a task starts.*
>
> ***Practice**: Install the CLI, verify its version and sign-in, then check desktop and mobile availability.*

To do a good job, one must first sharpen one's tools. In "Real-World Product Talk", I often emphasize a core principle: **The first step of AI-Native development is configuring your "cockpit" to be sufficiently stable.** Many beginners rush into prompting or writing code with AI, only to end up with AI screaming errors in the terminal and wasting tokens because of local environment mismatches or incorrect permission settings.

This chapter will guide you step-by-step through configuring Codex's multi-device productivity matrix, including the Codex CLI, the unified ChatGPT Desktop App (Code Mode), and mobile alerts when a notification workflow is configured.

A Way to Think About It: Your Three-Surface Mission Cockpit

Do not think of these surfaces as disconnected programs. Imagine commanding a space exploration ship:

```
【Codex CLI】                ──> Main Engine Room: Mounted in your terminal, handling high-throughput batching, tests, and CI builds.
【ChatGPT Desktop Code Mode】──> Bridge Tactical Screen: Multi-repo diff reviews, Sites in-place preview, and visual audits.
【ChatGPT Mobile App】       ──> Commander's Pager: Monitoring build statuses away from your desk, approving actions via Guardian.
```

Each surface handles its specialized duty, freeing you from being chained to a keyboard.

Practice: Start with Three Steps

You do not need complex network tunneling on day one. Follow these 3 steps to run your first task in 5 minutes:

1. **Step 1: Install the CLI and Authenticate**
   Check that Node.js and npm are installed, then run:

```
npm install -g @openai/codex@latest
codex
```

A browser window will open automatically. Sign in with your ChatGPT account to bind.

2. **Step 2: Check the Version and Choose a Current Model** Run `codex --version` first. At this edition's October 21, 2026 cutoff, the official changelog lists CLI `0.163.0`; your local version may differ. Changelog
   If your account has access, start with GPT-6.1 Sol without overwriting an existing `~/.codex/config.toml`:

   ```
   codex --version
   codex --model gpt-6.1-sol
   ```

   To make it the local default, set this key in your existing `~/.codex/config.toml`:

   ```
   model = "gpt-6.1-sol"
   ```

3. **Step 3: Run Your First Safe Sandbox Task**
   Navigate to your project directory and execute:

   ```
   codex exec --sandbox workspace-write "Inspect current code quality and suggest improvements"
   ```

2.1 CLI Setup & Version Checks

The core of Codex CLI is written in Rust (`codex-rs`), but OpenAI distributes it via npm. **As a user, you do NOT need a Rust compiler**, check the current package requirement with `npm view @openai/codex@latest engines`.

1. Installation & Environment Check

Run in your terminal:

```
# 1. Check Node.js and the current npm package requirement
node --version
npm view @openai/codex@latest engines

# 2. Install the current stable CLI; check the official changelog for its version
npm install -g @openai/codex@latest

# Or use the official install script (macOS / Linux)
curl -fsSL https://chatgpt.com/codex/install.sh | sh
```

Windows users can install via PowerShell:

```
powershell -ExecutionPolicy ByPass -c "irm https://chatgpt.com/codex/install.ps1 | iex"
```

Verify the installation:

```
codex --version
```

2. Authentication: ChatGPT Account vs. API Key

Codex CLI offers two authentication paths:

- **Option A (Recommended / Default)**: Run `codex` directly to log in via browser. **ChatGPT Plus ($20/mo), Pro, Team, Business, Edu, and Enterprise plans all include Codex quotas**, offering the highest ROI for solo developers.
- **Option B (Pay-as-you-go / CI)**: Use an OpenAI API Key for CI/CD pipelines or headless servers.

For API-key login, follow the Codex CLI documentation and load the secret from secure storage. Do not put a real key in the repository or a tutorial.

3. Billing Guardrails & Overrun Protection

1. **OpenAI Platform Hard Limit (Crucial)**: Set a monthly Usage Limit (e.g., $50 for beginners) in your OpenAI dashboard. Even in an infinite loop, your budget is safeguarded.
2. **Match model to task**: Select a model available in Codex; start complex daily coding with `gpt-6.1-sol` when available. Check API bills separately from Codex subscription usage.
3. **Sandbox and automatic review**: Use `--sandbox workspace-write` for new non-interactive calls. `--approve-for-me` is a separate review mode; it does not guarantee approval or have a documented fixed Luna reviewer. Permissions

4. Optional: Evaluate a Third-Party Provider Switcher

Before switching model providers, check the target API compatibility and how local data is handled. **Codex Switch** is a separate open-source tool maintained by this book's author, not an official OpenAI CLI feature. Its README documents supported versions, setup, and changes to local session files. Cross-provider resume is not guaranteed for future versions.

The current repository uses `codex-switch.sh`; it does not provide the `install.sh` or `codex-switch --to` commands previously shown here. Read the README and back up local sessions and configuration before setting up a provider. This command only checks the entry script after cloning; it does not switch providers:

```
git clone https://github.com/aipmer/codex-switch.git
cd codex-switch
test -f codex-switch.sh && sed -n '1,35p' README.md
```

2.2 Desktop Surface: ChatGPT Desktop App (Codex Code Mode)

> ⚠️ ***Major Ecosystem Milestone**: In late 2026, the standalone Codex Desktop App was officially retired and merged directly into the **ChatGPT Desktop App**.*

1. Installation & Accessing Code Mode

```
# macOS: Download ChatGPT.dmg and move to Applications
# Windows: Install via winget
winget install OpenAI.ChatGPT
```

Log in and click the **Codex** tab in the left sidebar to enter "Codex Code Mode".

New Productivity Capabilities:

- **Sites Hosting**: In-place web project creation, deployment, and preview with Annotations for point-and-click editing.
- **Multi-repo Review**: Inspect diffs across multiple folders in a single unified view.
- **Enhanced Browser**: Search browsing history directly and reference active Chrome tabs.

2. Computer Use Permissions (macOS)

To allow the agent to visually inspect your screen, grant two permissions under macOS "System Settings -> Privacy & Security":

```
[macOS System Settings] -> [Privacy & Security]
  ├─ Accessibility  ———> Check [ChatGPT] (Allows simulating clicks/keystrokes)
  └─ Screen Recording —> Check [ChatGPT] (Allows screenshot capture & Vision analysis)
```

Security Boundaries:

- You must explicitly approve each third-party application on first launch.
- ChatGPT cannot interact with Terminal itself, its own window, or `sudo` credential dialogs.

2.3 Mobile Sentinel: 24/7 Remote Oversight

1. **Native Task Syncing**: Signing into Codex with your ChatGPT account automatically mirrors running tasks to the ChatGPT Mobile App.
2. **Guardian Approvals & Alerts**: With Ch.08's mobile gateway, critical deployment gates trigger rich cards in Feishu or WeChat, enabling one-tap mobile approvals.

🛡️ Troubleshooting & Pitfall Cheat Sheet

Symptom	Root Cause	Instant Fix
`--full-auto` prints a deprecation warning	The old call remains a compatibility path	Use `--sandbox workspace-write` for new non-interactive runs
`SyntaxError: Unexpected token ...` (Node error)	Node.js version is older than 20	Run `node -v`; upgrade using `nvm use 20` or `nvm install 20`
`Permission denied: Screen Recording`	Missing macOS screen permissions	Toggle ChatGPT under "System Settings -> Privacy & Security -> Screen Recording" and restart
`gpt-5.4` is unavailable with Codex sign-in	It retired from Codex with ChatGPT sign-in on August 31, 2026	Select `gpt-6.1-sol` when your account has access

---

Current model and migration details · Codex tool guide

Ch.03 Diagnose Sandbox Access to Local Services

> ***Problem**: localhost inside an isolated environment may not refer to the host database.*
>
> ***Practice**: Locate the network boundary with a connection test, then configure a controlled tunnel if needed.*

When running database tests with Codex, the most frequent surprise for beginners is: while PostgreSQL runs perfectly in local Docker, the AI shouts `Connection refused to localhost:5432` in the terminal.

First identify whether the command runs on your host, in a container, or on a remote machine: `localhost` refers to a different machine in each case. This chapter tests the connection and adds controlled access only when needed.

A Way to Think About It: The Cleanroom Containment and Umbilical Pipeline

Think of Codex's runtime environment as a "high-level sterile containment cleanroom":

```
【Host Mac/PC】       ──> The Outside World: Holding your actual local database, secret keys, personal tools, and files.
【Codex Sandbox】     ──> The Sterile Cleanroom: Where the AI builds code and runs tests. Even if the code crashes, your host system is untouched.
【Reverse Tunneling】 ──> Umbilical Feed Pipeline: When the AI in the cleanroom needs to talk to the local host DB, you must bridge a dedicated pipe.
```

Without this pipe, the AI calling `localhost` inside the cleanroom reaches only bare walls, naturally throwing `Connection refused`.

Practice: Start with Three Steps

Follow these 3 steps to connect the sandbox with your local database:

1. **Step 1: Confirm Local Service is Actively Listening**
   Verify in terminal that Postgres is running locally on 5432:

   ```
   lsof -i :5432
   ```

2. **Step 2: Launch a Quick Port Tunnel (Ngrok or SSH)**
   Open a temporary public tunnel for port 5432:

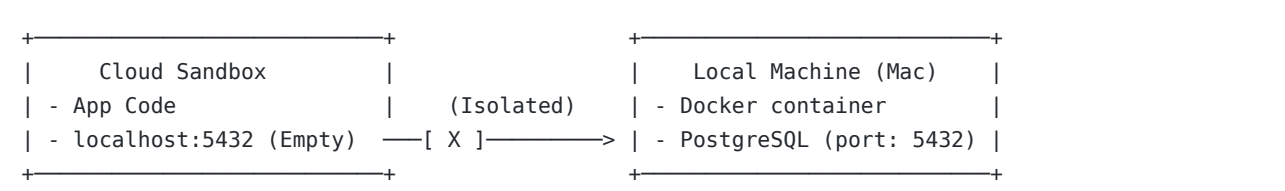
   ```
   ngrok tcp 5432
   # Output: tcp://0.tcp.ngrok.io:12345
   ```

3. **Step 3: Inject the Tunnel URL into Codex Task**
   Pass the tunnel address into the environment and run:

   ```
   export DATABASE_URL="postgresql://postgres:password@0.tcp.ngrok.io:12345/dev_db"
   codex exec --sandbox workspace-write "npm run test:db"
   ```

3.1 Sandbox Network Barriers: Why Can't `localhost` Connect?

By default, the sandbox and your host machine are network-isolated:



In current Codex CLI, sandbox boundaries are defined by the permission settings:

- `--sandbox read-only` : Analysis only, zero write permissions.
- `--sandbox workspace-write` : Recommended default; allows edits within the project workspace while protecting the host OS.

To connect with host databases or microservices, you must establish **port mapping and reverse tunneling**.

3.2 Practice: Setting Up Reverse Tunnels via SSH / Ngrok

Method 1: SSH Reverse Port Forwarding (Recommended for Long Term)

```
# Forward local port 5432 to port 54320 on your public VPS
ssh -R 54320:localhost:5432 user@your-public-vps.com -N
```

Then configure the sandbox:

```
export DATABASE_URL="postgresql://postgres:password@your-public-vps.com:54320/dev_db"
```

Method 2: Ngrok for Instant Tunneling (Zero-Server Option)

Run on host:

```
ngrok tcp 5432
```

Inject the printed `tcp://0.tcp.ngrok.io:12345` into Codex session.

3.3 Directory Mapping and Env Syncing

1. Security Isolation for Secrets

Never allow AI to commit raw `.env` files. Enforce in `.gitignore` and AGENTS.md:

```
## 🛑 Hard Constraints
- Never sync, copy, or commit files matching *.env.
- Always use `src/config.ts` or environment variables for secret injection.
```

2. Sandbox Cache Clearing

To avoid ghost build bugs, define clean run commands in AGENTS.md:

```
## 💻 Developer Commands
- **Clean Run**: `rm -rf node_modules/.cache .next/cache && npm run test`
```

🛡️ Troubleshooting & Pitfall Cheat Sheet

Symptom	Root Cause	Instant Fix
`Connection refused to 127.0.0.1:5432`	Sandbox attempted to query localhost inside cleanroom	Confirm tunnel is active; use the reverse tunnel address instead of `localhost`
`ngrok session expired / reconnecting`	Free ngrok tunnel timed out or changed port	Restart ngrok and copy new address, or use `scripts/codex-watchdog` helper
`EACCES: permission denied`	Agent tried to write to protected system directories	Ensure `--sandbox workspace-write` is used and edits stay within project root

---

Ch.04 Guide Agents with Goals and Acceptance

Checks

> ***Problem**: Vague requests invite wrong assumptions; overly detailed steps constrain implementation.*
>
> ***Practice**: Write a task with a goal, scope, constraints, and runnable acceptance checks.*

Regardless of model, a task that only lists steps is hard to evaluate without acceptance checks. When several implementations are valid, specify the goal, constraints, and verification before choosing the implementation.

This chapter shares how to guide Codex using a professional "Product Specs" approach in actual development.

A Way to Think About It: Michelin Chef and the Order Ticket

Many people dispatch tasks to AI like an unruly customer barging into a Michelin kitchen:

```
【Babysitting Commands (Process-Driven)】 ──> ❌ "Chef, take the knife in your left hand, chop
potatoes into 2mm strips, heat the oil to 180°C, stir for 3 minutes, then add 3g salt."
                                                (The chef feels insulted, and if stove pressure
shifts slightly, the dish burns.)
【Architect Spec Order (Goal-Driven)】    ──> ✅ "Chef, I need pan-seared crispy potatoes as a
steak side dish:
                                                - Goal: Crispy texture, dinner side dish;
                                                - Constraints: Under 200 kcal, strictly NO butter
or peanuts (customer allergy);
                                                - Validation: Plated within 15 minutes, core
temperature at 75°C."
```

Codex is an algorithmic master chef. Your role is defining what to make, what cannot be touched, and how success is measured—leaving the heat and chopping technique to the agent.

Practice: Start with Three Steps

Draft your first Goal-Driven Spec in 3 steps:

1. **Step 1: Define the End State (Goal)**
   State the deliverable in one sentence: "Implement Stripe payment session creation at `/api/checkout`".
2. **Step 2: Enforce Inviolable Guardrails (Constraints)**
   List 2-3 boundaries: "Never log secret keys in plain text; do not modify global middleware".
3. **Step 3: Provide Automated Verification Assertions (Validation)**
   Give a runnable command: "Running `npm test -- checkout.test.ts` must pass 100% on the first run".

4.1 The Goal-Driven Markdown Specs Template

When triggering coding tasks in your terminal or ChatGPT Desktop, use this structured Markdown layout:

```
# 🎯 Goal
[Describe desired business outcome. Example: Implement GitHub OAuth login route and persist
preferences.]

# 🛑 Constraints
- [Security: Never commit credentials or write keys to log streams.]
- [Stack: Strictly use existing Tailwind classes; do not introduce new UI libraries.]
- [Anti-pollution: Prohibited from editing any file under src/legacy/.]
```

```
# 🧪 Validation Specs
- [Automated Testing: Running `npm run test:unit` must pass 100%.]
- [Edge behaviors: If inputs are missing, return 400 Bad Request with structured JSON error payloads.]
```

4.2 Real Case Comparison: Traditional Prompt vs. Goal-Driven Specs

Suppose we need to write a **"Redis-based Rate-Limiting API Proxy Service"**.

❌ Traditional Process-Driven Prompt

> "*Please help me write an API proxy with Express. First, import express and express-rate-limit. Then configure rate-limiting, setting windowMs to 15 minutes and max to 100. Then write a route `/api/proxy` using axios to request the third-party API `https://api.github.com`. If successful, return the data; if it fails, return a 500 error. Make sure to include the Authorization Bearer Token in the headers.*"

✅ Goal-Driven Specs

```
# 🎯 Goal
Implement an Express API proxy route that forwards all incoming requests safely to the GitHub API.

# 🛑 Constraints
- Must use Redis as the rate-limiting data source (no memory-based limiting) to support multi-container scaling.
- Limit proxy request timeout strictly to 3000ms to prevent hanging the Node event loop.
- Never write the GitHub token into the codebase or logs; fetch securely from `process.env.GH_TOKEN`.

# 🧪 Validation Specs
- Under simulated load: return 429 Too Many Requests when requests exceed 60 req/min.
- Fault tolerance: return 504 Gateway Timeout with structured JSON on timeouts or network drops.
```

4.3 GPT-5.6 Terra Reasoning Architecture for Specs

When presented with this spec, Codex breaks down the solution chain:

```
[Parse Specs Goal] ──> Analyze Constraints (Redis rate-limit / 3s timeout / Secret isolation)
                                  │
                                  ▼
[Self-Directed Architecture] ──> Import ioredis + rate-limit-redis, configure Axios abort controllers
                                  │
                                  ▼
[Align with Validation Specs] ──> Simulate 429 rate limit & timeout fallbacks in unit tests
                                  │
                                  ▼
[Self-Heal Code Until Tests Pass 100%] ──> Deliver clean code with green verification reports
```

Delegate logic planning to AI, but keep verification standards firmly in your own hands.

🛡️ Troubleshooting & Pitfall Cheat Sheet

Symptom	Root Cause	Instant Fix
Overly fragmented steps cause agent to freeze on errors	Procedural commands stripped GPT-5.6 of self-healing planning space	Strip out implementation steps; retain only Goal, Constraints, and Validation
Agent modified unrelated files, breaking other features	Lacked directory scope constraints	Declare strict boundaries: "Edits restricted to `src/modules/auth/`; never touch other directories"
Agent claimed completion, but logic has fatal flaws	Missing automated validation assertions	Add explicit requirement: "Must run `npm test` and produce green unit test assertions"

Ch.05 Record Project Rules in AGENTS.md

> ***Problem**: Without project guidance, an agent may edit unrelated files or retry failed commands.*
>
> ***Practice**: Create a maintainable AGENTS.md and distinguish written guidance from actual permissions and sandbox controls.*

In the product development process, one of the most frustrating scenarios is fixing one bug only to introduce three new ones, or coding a new feature while completely disregarding the team's established coding standards.

In human-machine collaborative development, place an **`AGENTS.md`** file in the project root to share the stack, commands, and collaboration rules. Use permissions, review, and tests to control consequential operations.

This book calls the format **Codex Collaboration Protocol (CAP)**. It is project guidance, not an enforced Codex security mechanism.

A Way to Think About It: The New Employee Onboarding Handbook & Safety Code

Think of Codex as a brilliant, hyper-energetic new intern on their first day:

```
【Without AGENTS.md】 ──> The intern arrives with no rulebook. To fix a minor frontend alignment bug,
                         they casually rewrite your 3-year-old core authentication module into an esoteric syntax,
                         crashing production upon release.
【With AGENTS.md】    ──> The intern sits at their desk and reads the team's printed handbook:
                         - "This is Next.js 15 with Tailwind" (Explicit stack)
                         - "Build with `npm run build`, never alter standard scripts" (Normalized operations)
                         - "DO NOT touch the auth/ directory under any circumstance" (Absolute red lines)
                         The intern immediately grasps the boundaries and operates with peak efficiency without breaking a thing.
```

`AGENTS.md` is that zero-friction onboarding manual for AI agents.

Practice: Start with Three Steps

Start your first project rules file with these three steps:

1. **Step 1: Create the File in Your Project Root**
   Run in your terminal:

   ```
   touch AGENTS.md
   ```

2. **Step 2: Paste the 4-Section Minimal Scaffold**
   Save the following into `AGENTS.md`:

   ```
   # 🤖 Codex Collaboration Protocol

   ## 📌 Project Fingerprint
   - Stack: Next.js 15, TypeScript, Tailwind CSS

   ## 💻 Developer Commands
   - Build: `npm run build`
   - Test: `npm run test`

   ## 🛑 Hard Constraints
   - Never update package.json dependencies without human confirmation.
   - Never touch files inside `src/legacy/`.
   - Always run `npm run build` before completing the task.
   ```

3. **Step 3: Launch Codex to Verify Protocol Loading**
   Run `codex`; the initialization logs will confirm that workspace rules have been loaded.

5.1 Why Do We Need `AGENTS.md`?

`AGENTS.md` is OpenAI Codex's official instruction specification file. When starting up, Codex scans and merges configurations hierarchically:

1. `~/.codex/AGENTS.override.md` (Highest priority, user overrides)
2. `~/.codex/AGENTS.md` (Global user instructions)
3. Project root `AGENTS.md` (Team standard)
4. Current directory `AGENTS.md` (Specific workspace scope)

Its core value lies in:

1. **Immediate Context Restore**: Every time Codex starts, its first action is scanning `AGENTS.md`, instantly picking up the project's tech stack, directory structure, and collaboration constraints.
2. **Anti-Corruption Layer**: It explicitly defines which directories and files are "read-only/off-limits," preventing Codex from unilaterally refactoring critical security modules.
3. **Command Execution Guardrails**: It specifies permissible test and deployment command options, avoiding destructive shell commands.

5.2 The Four Core Sections of `AGENTS.md`

```
# Project Fingerprint
- Tells the AI what kind of project this is and its core tech stack.

# Developer Commands
- Explicitly states the command lines for building, testing, and running migrations.

# Styles & Architecture Patterns
```

```
- Dictates where files should be placed, size limits, and design patterns.

# Agent Boundary & Hard Rules
- Defines absolute forbidden paths. If touched, Codex must abort and ask for human verification.
```

5.3 Practice: A Production-Grade `AGENTS.md` Template

```
# 🤖 Project: Aurora SaaS Core (AGENTS.md)

## 🧬 Project Fingerprint
- **Stack**: Next.js 15 (App Router), TypeScript, Prisma ORM, TailwindCSS.
- **Database**: PostgreSQL on Supabase.
- **Auth**: Next-Auth v5.

## 💻 Developer Commands
- **Install**: `npm install` (Only run if package.json has changed)
- **Dev Server**: `npm run dev`
- **Lint Code**: `npm run lint`
- **Run Tests**: `npm run test`
- **DB Migration**: `npx prisma migrate dev` (Never run in production branch)

## 🎨 Styles & Architecture Patterns
- **Directory Structure**:
  - Components: Keep UI components inside `@/components/ui/` (Shadcn styled).
  - Business Logic: Custom React hooks must go to `@/hooks/`.
  - API Routes: Next.js Route Handlers go to `src/app/api/.../route.ts`.
- **Formatting**:
  - Keep components modular. If a component exceeds 200 lines, decompose it.
  - All API routes must implement Zod schema validation for request body.
  - Return HTTP 400 for validation errors, 500 for internal uncaught errors.

## 🛑 Agent Boundary & Hard Rules
- **READ-ONLY Directories**:
  - Never modify files inside `src/app/api/auth/[...nextauth]` (OAuth Core).
  - Never alter `prisma/schema.prisma` without explicit human confirmation.
- **PR Rules**:
  - Before declaring a feature complete, run `npm run test` and `npm run lint`.
  - If tests fail, rollback the change immediately and report the error logs.
- **Security Check**:
  - Never commit raw `.env` files or API Keys. Use environment variables.
```

5.4 Rules, Sandboxes, and Hooks Have Different Jobs

`AGENTS.md` supplies project context and behavioral instructions; it does not replace file-system permissions. Use the Codex sandbox to restrict writable paths. Use human approval or `--approve-for-me` to review eligible escalation requests. Hooks can run checks at supported events, but review their source and trust status first. Permissions · Hooks docs

```
codex --sandbox workspace-write "Review the current changes and report issues"
```

This configuration selects GPT-6 Sol and checks the current repository for whitespace errors at the end of a turn. Both the `Stop` event and its handler use array syntax. Verify trust with `/hooks` after enabling:

```toml
# ~/.codex/config.toml
model = "gpt-6-sol"

[[hooks.Stop]]

[[hooks.Stop.hooks]]
type = "command"
command = "git diff --check"
```

Model selection, sandbox permissions, and Hook checks are independent settings. A Hook check does not replace builds or tests.

🛡️ Troubleshooting & Pitfall Cheat Sheet

Symptom	Root Cause	Instant Fix
Agent occasionally breaks rules despite being written	Rules too long, contradictory, or buried in verbose prose	Keep rules concise and prefer negative constraints ("Never do X"); elevate to Hooks/Sandbox
AI falls into a 5-attempt self-correction death loop	Missing Anti-Loop circuit breaker in instructions	Add to AGENTS.md: "Stop immediately after 2 consecutive failed attempts and report root causes"
Agent installs arbitrary unfamiliar npm packages	Package managers allowed without gating	Mandate in Hard Rules: "Never run `npm install` for new dependencies without human consent"

Ch.06 Inspect Progress and Correct Course

> ***Problem**: An incorrect assumption can expand the scope of a running task.*
>
> ***Practice**: Inspect visible plans, commands, and diffs; interrupt and restate a concrete constraint when needed.*

In traditional software development, when managing a junior engineer, your biggest fear is having them work in isolation for a week, only to deliver code that completely deviates from business goals or breaks the main branch.

With any model, an incorrect assumption or task boundary can lead to unnecessary edits. Check visible plans, tool calls, and diffs during long tasks.

This chapter uses observable execution records to spot drift and restate concrete constraints.

A Way to Think About It: An "Exam Proctor Window" into AI's Mind

Never treat AI reasoning as a black box:

```
[Passive Blind Waiting] ──> Like waiting outside an exam hall for 2 hours, only to discover the
student misremembered the fundamental formula on Question 1, failing the entire exam.
[Proctor Supervision]   ──> Like standing right behind the student, observing their draft
scratchpad (Reasoning Summary):
                            - The student drafts: "Assume we need to rewrite the entire database
schema..."
                            - You tap their shoulder immediately: "Don't touch the schema, just
```

```
add a query index!"
                              - The student strikes out the wrong idea and returns to the right
track.
```

Reading the Reasoning Summary gives you real-time visibility into that draft scratchpad.

Practice: Start with Three Steps

Learn to supervise and correct the AI like a tech lead in 3 simple steps:

1. **Step 1: Launch the Interactive TUI and Monitor Reasoning**
   Run `codex` directly in your terminal and observe the rolling thought summary in the split pane.
2. **Step 2: Press `Ctrl + C` the Instant an Assumption Derails**
   The moment you see the AI planning to pull in unfamiliar external packages or refactor core untouched files, hit `Ctrl + C` immediately.
3. **Step 3: Correct with One Precise Sentence and Resume**
   Type your constraint directly: "*Do not rewrite existing tables; solve this with in-memory caching.*" Ask the agent to restate the new constraint before it continues editing.

6.1 Why Inspect the Model's Reasoning Process?

The biggest difference between deep-reasoning models (such as GPT-5.6 Terra) and traditional models is that before producing final code, the reasoning model conducts in-depth internal hypothesis testing and simulation. The Codex TUI displays this process in real time as a **Reasoning Summary**.

```
[User Request] ──> 1. Parse Goals & Constraints ──> 2. Plan Steps ──> 3. Run Tests ──> 4. Self-
Correct ──> [Final Output]
                     └──────(Displayed in TUI as Reasoning Summary: Your Proctor
Viewport)──────┘
```

> ⚠️ ***Note**: The full internal Chain of Thought (CoT) is guarded for safety; what you see in the TUI is a **high-fidelity reasoning summary** generated by the model. This summary is more than sufficient to evaluate technical decisions and architectural directions.*

6.2 How to Observe and Interpret Reasoning in the TUI

1. Interactive Mode (TUI)

Launch `codex` to enter the interactive terminal interface, where the main pane streams the reasoning summary. Essential inspection and control commands:

```
# Common TUI shortcut commands
/diff       # Review actual code modifications in the current session
/review     # Summon a reviewer subagent to audit recent changes
/copy       # Quickly copy the last generated response or code snippet
```

2. Non-Interactive Mode (Scripted Pipelines)

When running background tasks or long scripts, stream structured JSONL logs:

```
# Non-interactive execution, streaming structured JSONL
codex exec --json "Refactor auth middleware" > task.jsonl
```

Healthy Reasoning Stream Example

When Codex receives the task "*Fix connection timeout in the Redis rate limiter,*" a healthy reasoning stream looks like this:

```
[Reasoning Summary - Normal Flow]
- User wants to fix Redis rate limiter connection timeout.
- Checking existing implementation in src/lib/redis.ts...
- Found `redis = new Redis()` without retryStrategy.
- If Redis is unreachable, this hangs the Node process, violating the 3000ms SLA in AGENTS.md.
- Action Plan:
  1. Add `maxRetriesPerRequest: null` and explicit connectTimeout: 2000.
  2. Implement custom retryStrategy up to 3 attempts.
  3. Run `npm run test:redis` to verify behavior.
```

6.3 Identifying Typical AI "Death Loops"

Stay alert for these two classic self-spinning traps during daily development:

1. The Dependency Loop

- **Symptom**: The AI attempts to install an unvetted library. When installation fails, it tries three different versions or switches to another even more obscure library.
- **Signal**: Repeated occurrences of `npm install --legacy-peer-deps` more than twice in the terminal.

2. The Regression Loop (Whack-a-Mole)

- **Symptom**: Editing file A breaks test B; it modifies test B, triggering errors in module C; it edits C, which breaks file A again.
- **Signal**: Test pass rates fluctuating between 80% and 90%, with repeated edits to the exact same set of files.

6.4 The Three-Step Intervention: Interrupt, Correct, and Take Over

Step 1: Decisive Interruption (`Ctrl + C`)

Press `Ctrl + C` to stop the current run immediately and avoid token waste.

Step 2: Point-to-Point Correction (Direct Dialogue)

Directly point out the blind spot:

```
You were trying to install axios-retry, but this project strictly forbids external HTTP retry
libraries. Use native AbortController for timeout handling and replan.
```

Step 3: Human Takeover and Git Rollback

If the edit is wrong, inspect the diff and confirm there are no human changes to keep before restoring only the target file:

```
git diff -- src/lib/redis.ts
git restore -- src/lib/redis.ts
```

Append a guardrail into AGENTS.md: "*Strictly prohibit external retry libraries.*"

6.5 Companion Hands-on Sandbox: Codex Web Copilot (Chrome Extension)

To experience how to steer reasoning chains and enforce anti-loop guardrails during browser extension development, this repository provides a ready-to-run sandbox:

👉 **Source Sandbox Directory**: examples/ch06-chrome-extension

Core Highlights:

1. **Pure Native Manifest V3**: Zero build bundling dependencies. Load unpacked in Chrome Developer Mode in under 1 minute;
2. **Strict CSP Guardrails**: `AGENTS.md` forbids inline scripts and `eval()`, showing how AI agents write compliant code under strict browser sandboxes;
3. **Automated Guard Tests**: Run `npm test` to automatically validate MV3 specs and script syntax.

🛡️ Troubleshooting & Pitfall Cheat Sheet

Common Pitfall	Root Cause	Rapid Diagnosis & Fix Guide
AI stuck in an endless loop of self-edits	Prompt lacks an explicit retry cap	Hit `Ctrl + C` immediately, then instruct: "*Retry threshold reached. Stop spinning and output your diagnostic findings.*"
Interrupting and re-prompting loses previous context	Session context was dropped	Use `codex resume` to restore the original session thread and preserve reasoning context
AI broke multiple legacy core files	Changes were not isolated	Inspect `git status` and `git diff`, preserve any human edits, then restore only confirmed unwanted changes; use a separate branch or worktree next time

---

Ch.07 Validate Frontends in a Real UI

> ***Problem**: Builds and unit tests cannot catch every overlap, overflow, or click failure.*
>
> ***Practice**: With the required tools and permissions, check viewport sizes, interactions, and screenshots.*

In traditional UI fidelity reviews, the most time-consuming task for product managers and frontend developers is "pixel-eye" alignment verification:

“This button seems shifted 4 pixels to the left.”
“This popup gets obscured by the virtual keyboard on mobile dimensions.”

In the 2026 Codex ecosystem, combining **ChatGPT Desktop (Codex Code Mode)** with OpenAI's latest frontier flagship **GPT-6 Astra** (which supports Computer Use workflows through tools), the agent can not only write code but also "use eyes and hands" to directly operate your macOS desktop: launching browsers, adjusting developer tool resolutions, and performing high-fidelity visual audits.

This chapter teaches you how to orchestrate Computer Use to automate design-fidelity inspection for frontend UI.

A Way to Think About It: A 24/7 Tireless "Pixel Quality Inspector"

Think of Computer Use as hiring a dedicated UI QA inspector sitting right beside your desk:

```
[Manual UI Inspection] ──> Holding Figma mockups in one hand, switching Chrome tabs with the
other,
                           squinting to verify font sizes, resizing windows manually for
breakpoints,
                           and manually refreshing pages after every CSS tweak (slow and
exhausting).
[Computer Use Mode]    ──> You hand the inspector a task card: "Review /auth/login against
design mockup."
                           The inspector puts on blue-light glasses (screenshot analysis),
                           takes the mouse (simulated clicks), measures a 16px offset,
                           updates the Tailwind classes in the editor, refreshes the browser,
                           and captures a clean screenshot to report completion.
```

Automated checks help locate UI problems; people still judge interaction quality and the final design.

Practice: Start with Three Steps

Run your first AI visual inspection in 3 simple steps:

1. **Step 1: Verify ChatGPT Desktop Permissions**
   In macOS **System Settings → Privacy & Security**, ensure that **ChatGPT** is granted **Accessibility** and **Screen Recording** permissions.
2. **Step 2: Start Your Local Frontend Server**
   Start your development server in the terminal (confirming it opens in your browser):

   ```
   npm run dev
   # Verify http://localhost:3000 is accessible
   ```

3. **Step 3: Summon `@Chrome` in Code Mode with an Inspection Task**
   In the ChatGPT Desktop Codex mode prompt, enter:

   ```
   @Chrome Open http://localhost:3000/auth/login, capture the main card area, check for
   horizontal viewport overflow, and adjust padding.
   ```

7.1 Safety First: Sandbox Boundaries and Application Whitelisting

Allowing an AI to operate your physical screen requires strict security fencing. To prevent the agent from accidentally clicking personal messaging apps or altering system files due to misrecognition, establish **application-level whitelisting**.

1. GUI Permission Whitelist

In the ChatGPT Desktop client:

- The first time Computer Use interacts with a specific application (e.g., Google Chrome), a system prompt will ask for confirmation;
- You can choose **“Just this once”** or **“Always allow”**;
- Strictly avoid whitelisting personal apps (e.g., WeChat, Slack, Mail, or the terminal itself);
- **Factory Hard Bounds**: Codex cannot operate the terminal, the ChatGPT client itself, or system administrative privilege dialogs (`sudo`), forming an OS-level hard isolation barrier.

2. Coordinate Positioning and Visual Perception

Computer Use operates via a robust perceptual feedback loop:

```
[Viewport Screenshot] ──> [GPT-5.6 Vision Analysis] ──> [Compute Pixel Coordinates (x:450, y:230)] ──> [Execute Mouse Click/Drag]
```

Combined with the 2026 **Appshots** feature (double-tap shortcut), visual snapshots and text contexts of the focused foreground window can be instantly injected into the conversation without manual screenshot uploads.

7.2 Practical Visual-Driven UI Review: Figma Mockup Alignment

This is the most practical automated scenario: **having Codex autonomously compare design mockups with local web rendering, auto-tuning styles.**

🎯 Goal

Compare local page `/auth/login` with design mockup screenshot `figma_login_mockup.png`, eliminating spacing discrepancies.

🛑 Constraints

- Only Tailwind utility class adjustments in `src/app/login/page.tsx` are permitted.
- Modifying existing DOM trees or semantic tags is prohibited.

🧪 Automated Execution Specs

Submit this prompt in the ChatGPT Desktop client:

```
@Chrome Complete the UI visual inspection following these steps:

# 🎯 Goal
Compare and align browser rendering with figma_login_mockup.png.

# 🛑 Constraints
- Only use Tailwind utility classes in src/app/login/page.tsx.
- Do not change the DOM structure.

# 🚀 Execution Steps
1. Open Google Chrome and navigate to http://localhost:3000/auth/login.
2. Capture screenshot of the login form container.
3. Compare against assets/figma_login_mockup.png and identify padding discrepancies.
4. Update Tailwind classes in src/app/login/page.tsx to match spacing.
5. Reload page and confirm visual alignment within 2% delta.
```

7.3 Conceptual Execution Flow of Codex Automated Auditing

```
> Running visual review for /auth/login
> Step 1: Opening Google Chrome on http://localhost:3000/auth/login...
> Step 2: Taking screenshot. Saved to /tmp/screenshot_v1.png
> Step 3: Calling vision model for image comparison.
    Analysis: "Login card padding-top is 16px (pt-4), but mockup requires 32px (pt-8). Font size is text-base, needs text-xl."
> Step 4: Updating src/app/login/page.tsx via apply_patch...
> Step 5: Reloading Chrome tab and taking validation screenshot...
> Step 6: Vision check: "Visual delta is within 1.2% tolerance. Perfectly aligned."
> Task completed successfully.
```

7.4 Advanced: Sites In-Place Preview and Multi-Device Responsive Auditing

Leveraging the 2026 ChatGPT Desktop **Sites** feature alongside mobile breakpoint audits:

```
@Chrome
# 📱 Mobile Viewport Inspection
1. Open Chrome DevTools.
2. Toggle Device Toolbar and select iPhone 15 Pro (393 x 852).
3. Verify the submit button does not drop below the first screen fold.
4. If obscured, reduce hero section padding to keep the CTA button immediately clickable.
```

Check pages at several viewport sizes and through real interactions. Record layout and usability issues, then adjust the styles based on what you observed.

7.5 Cross-Domain Extension: From Web Audits to Autonomous 3D Software Control (Blender Integration)

GPT-6 Astra can participate in desktop workflows when the client and tools support them. The Blender steps below are this book's practice example and require checking actual permissions and application state; they do not rely on an unverified 3D benchmark figure. Official GPT-6 guide

In a desktop environment, agents can autonomously interact with professional 3D creative suites like **Blender**:

- **Viewport Operations & Shading Checks**: The agent uses Computer Use to toggle viewports between Shading material preview and Rendered modes;
- **Inspecting Geometry & Shader Artifacts**: The agent takes viewport screenshots to identify inverted surface normals, missing UV maps, or washed-out lighting;
- **Spatial Pipeline Synergy**: Can be combined with the **Tripo3D + Blender + Astra** example workflow detailed in Ch.13, linking prompts, geometry generation, and render checks in an inspectable workflow.

🛡️ Troubleshooting & Pitfall Cheat Sheet

Common Pitfall	Root Cause	Rapid Diagnosis & Fix Guide
`Computer Use is not supported on this platform`	OS is not macOS or user is in an unsupported region	Ensure macOS client is used, or switch to a Headless Puppeteer script approach as an alternative
`Failed to capture window: Permission denied`	macOS Screen Recording permissions missing or needs refresh	Open System Settings, toggle ChatGPT's "Screen Recording" permission off and on, and restart the client
AI mouse clicks wild coordinates off target	Browser zoom is not 100% or multi-display DPI scaling mismatch	Reset Chrome zoom to standard 100%, and position the target test window in the center of the primary display

---

Ch.08 Route Build Alerts and Approvals to Mobile

> ***Problem***: *Build failures or pending approvals can go unnoticed away from the desk.*
>
> ***Practice***: *Set up an alert and human approval flow, then verify delivery, identity, and outcomes.*

Build outcomes and pending approvals still need attention when you leave your computer. This chapter connects alerts and separates automatic review from human decisions.

The example sends GitHub Actions failures to Feishu and demonstrates a local decision relay. `--approve-for-me` reviews eligible permission requests; production releases still require separate checks of project, version, and deployment result.

A Way to Think About It: Alerts and Approvals Are Separate Flows

Automatic checks find failures, notifications bring results to your phone, and an authorized system must handle approvals:

```
GitHub Actions check fails ──> Feishu alert ──> Review the run log on mobile
Codex permission request ──> Current sandbox and approval policy ──> Human review if required
Production deploy request ──> Verify project, version and approver ──> Separate deploy and
result check
```

Configure and verify each flow separately. Receiving an alert or recording approval does not mean a deployment succeeded.

Practice: Start with Three Steps

Set up your mobile alert notification channel in 3 simple steps:

1. **Step 1: Create a Feishu Custom Bot** Add a bot in your group chat settings and copy its incoming Webhook URL.
2. **Step 2: Send a Test Alert Card from Terminal**
   Set `FEISHU_WEBHOOK_URL` in your terminal, then use curl to test notification delivery:

   ```
   curl -X POST -H "Content-Type: application/json" \
     -d '{"msg_type":"text","content":{"text":"Codex test alert: please confirm this message
   reaches your phone."}}' \
     "$FEISHU_WEBHOOK_URL"
   ```

3. **Step 3: Run a Check in the Sandbox** After confirming the tests do not modify production data, run the task in a workspace sandbox:

   ```
   codex exec --sandbox workspace-write --approve-for-me "Run full regression suite and
   refactor legacy types"
   ```

8.1 Two-Tier Sentinel Dispatch Architecture

The diagram shows a suggested routing pattern. Automatic review depends on the actual permission policy; a phone notification does not itself grant deployment permission:

```
[Project checks] ──(failure)──> [Feishu alert] ──> [Review logs on mobile]
[Codex permission request] ──> [Current sandbox and approval policy] ──> [Human handles pending
requests]
[Deployment request] ──> [Identity and task checks] ──> [Human approval] ──> [Separate
deployment verification]
```

Companion open-source references:

- Related reference project: plugins-codex-feishu
- Minimal Local Tunnel Gateway: scripts/codex-watchdog

8.2 Practice: GitHub Actions Build Failures and Webhook Setup

In a Node.js project with an `npm test` script, create `.github/workflows/codex-watchdog.yml`. Save the Feishu bot URL as a repository secret named `FEISHU_WEBHOOK_URL`. This sends a short alert when checks fail and keeps the run marked as failed; inspect Actions logs for details:

```yaml
name: Codex Agent Watchdog

on:
  push:
    branches: [ main ]
  workflow_dispatch:

jobs:
  agent-build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20

      - name: Run Project Checks
        id: check
        continue-on-error: true
        run: |
          npm ci
          npm test

      - name: Push Fail Notice to Mobile
        if: steps.check.outcome == 'failure'
        env:
          FEISHU_WEBHOOK_URL: ${{ secrets.FEISHU_WEBHOOK_URL }}
        run: |
          curl -fsS -X POST -H 'Content-Type: application/json' \
            -d '{"msg_type":"text","content":{"text":"Project checks failed: open GitHub Actions to review this run."}}' \
            "$FEISHU_WEBHOOK_URL"

      - name: Keep Check Failure Visible
        if: steps.check.outcome == 'failure'
        run: exit 1
```

8.3 Mobile Bidirectional Interaction and Remote Approval

1. Scenario: Production Deployment Approval Gate

This illustrates a notification that would require a separate approval system. The preceding Actions alert and local relay do not send this card automatically:

```
🚨 [Codex Auth Requested]
Project: pmer-cn-saas
Action: Deploy to production (Vercel)
Change summary: Implemented Stripe subscription webhook in /api/stripe.
Tests: 12 passed, 0 failed.
[Directive Command]: Reply "1" to approve deployment, "0" to record a denial; review any further
action manually.
```

2. Check the Local Decision Relay

The companion `scripts/codex-watchdog/lib/gateway.js` only records a decision in a temporary signal file. **It does not deploy, terminate Codex, or roll back Git.** It binds to `127.0.0.1` and requires a random token. A Feishu integration needs an authenticated forwarding layer; the deployment controller must also validate project, task, and decision age.

```
cd scripts/codex-watchdog
npm ci
export CODEX_WATCHDOG_TOKEN="$(openssl rand -hex 32)"
node bin/cli.js gateway --port 8080 &
gateway_pid=$!
```

In the same terminal, send a local test request. A `denied` response means the decision was recorded; no files are deleted. Stop the relay after the check:

```
curl -fsS http://127.0.0.1:8080/api/mobile-reply \
  -H "Authorization: Bearer $CODEX_WATCHDOG_TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"userMessage":"0"}'
kill "$gateway_pid"
```

8.4 Related Reference Project: Codex Feishu Sentinel

The author-maintained **plugins-codex-feishu (Feishu Assistant)** repository offers a reference for Feishu notifications and approvals. Check its current README, required permissions, and test results before use.

Check these capabilities separately when relevant:

1. **Daily Inspection & Sentinel Reports (Quick Start)**:
   - Check the report's data source, recipient, and schedule, then send it to a test chat.
2. **Mobile Alerts & Two-Way Approval (Offline Orchestration)**:
   - Check what triggers alerts, whether they arrive, and how mobile replies are received.
   - Submit a decision on mobile; the deployment controller must check identity, project, task, and expiration. Denial does not automatically roll back code.
3. **Knowledge Base Persistence (Team Collaboration)**:
   - If you need Feishu documents or Bitable, verify app scopes and actual writes first.

> ***Before setup**: If the companion project provides a Feishu app manifest, still check its scopes, event subscriptions, and connection settings in the Feishu Open Platform, then test the alert in a non-production chat.*

🛡️ Troubleshooting & Pitfall Cheat Sheet

Common Pitfall	Root Cause	Rapid Diagnosis & Fix Guide
Phone does not receive Webhook messages	Bot security settings require keyword or IP whitelist	In bot settings under "Security", add matching keyword (e.g., `Codex`) or configure request signature
Replied "1" but nothing deployed	Local relay only records a decision	Check the authenticated forwarding layer, task validation, and separate deployment controller
Phone receives repeated approval messages	The same request is sent again	Give requests unique IDs, deduplicate them, and set an expiration time

---

8.5 Keep Notifications and Confirmation Human

Running repeatable checks in CI and sending failures to a phone can reduce time spent waiting at a desk. Production releases and data changes still need an authorized person to review the specific request and verify the outcome.

Ch.09 Baseline and Refactor Legacy Code Incrementally

> ***Problem**: Without tests, it is hard to tell whether a change broke legacy behavior.*
>
> ***Practice**: Map dependencies and test critical paths before refactoring in small commits.*

When working on solo projects or inheriting legacy codebases, the biggest source of anxiety is taking over undocumented, test-free "spaghetti code" left behind by previous teams. Any minor code change can detonate hidden mines buried deep in the system.

First map entry points, dependencies, and critical behavior. Repeatable tests protect existing functionality; a large model context does not replace incremental verification.

A Way to Think About It: Changing Tires on a Speeding Truck on the Highway

Refactoring an active legacy system is like this high-wire operation:

```
[Reckless Full Rewrite] ──> ❌ "This old truck is too beat-up; I'll build a brand-new truck on the shoulder!"
                            (The new build stalls for 6 months, while the business starves to death.)
[Progressive Decoupling] ──> ✅ The old truck continues hauling freight on the highway (zero business interruption):
                            1. Fly a drone under the chassis (AI scans full repository to generate a topology map);
                            2. Mount anti-rollover safety brackets (write behavioral snapshot tests for critical APIs);
                            3. Unscrew one rusty bolt at a time, test-drive it, and only proceed once verified green.
```

Codex acts as a millimeter-precision robotic surgical arm: as long as you wrap it in test guardrails, it excises dead code without nicking healthy tissue.

Practice: Start with Three Steps

When taking over an unfamiliar, messy repository on Day 1, take these 3 safe steps:

1. **Step 1: Ask AI for a Full-Repo Topology Map (Read-Only Exploration)**
   Run a read-only sandboxed analysis to output a Mermaid dependency graph:

   ```
   codex exec --sandbox read-only "Analyze architecture and database models; output Mermaid dependency diagram in docs/topology.md"
   ```

2. **Step 2: Capture a "Behavioral Snapshot" for Target Endpoints (Baseline Recording)**
   Instruct Codex: "*Do not touch business code in `src/api/` ; only write 3 unit tests covering success and failure paths for the existing endpoints.*"
3. **Step 3: Single-Point Extraction Guarded by Test Assertions**
   Issue minimally invasive refactoring specs: "*Extract only one function at a time, running `npm test` automatically after each modification. All tests must stay 100% green.*"

9.1 Step 1: Panoramic Reverse Engineering - Generating Codebase Topology

The primary duty when taking over a chaotic project is **drawing the map**. Leverage GPT-5.6 Terra's long-context scanning to directly reverse-engineer Mermaid Entity-Relationship Diagrams (ERD) and route topologies:

```
# 🎯 Goal
Analyze the database configuration and routing layer of the current project, generating a Mermaid ERD illustrating core table structures and foreign key relationships.

# 🛑 Constraints
- Only analyze prisma/schema.prisma or src/db/models/.
- Exclude temporary cache tables and third-party integration logs.

# 🧪 Validation Specs
- Output a syntactically valid Mermaid diagram in docs/database_topology.md.
```

Within seconds, Codex unpacks complex many-to-many relationships and foreign key cascades, far outstripping manual code audits.

9.2 Step 2: Safety Guardrails - Writing Behavioral Baseline Tests

The absolute golden rule of legacy code refactoring is: **Before performing surgery, hook up the vital-signs monitor (baseline tests).**

```
# 🎯 Goal
Write baseline unit tests for src/pages/api/checkout.ts, capturing real request inputs and response snapshots.

# 🛑 Constraints
- Strictly forbid altering any logic inside src/pages/api/checkout.ts itself.
- Use local Jest / Vitest to execute the test suite.
```

```
# 🧪 Validation Specs
- Cover three core branches: successful checkout, out-of-stock error, and unauthenticated access rejection.
- Ensure baseline tests pass 100% against the current legacy code.
```

9.3 Step 3: Minimally Invasive Surgery - Implementing Progressive Decoupling

1. Decoupling Fat Controllers

Gradually extract discount calculations, authentication, and inventory updates out of multi-hundred-line controller files into pure functions:

```
# 🎯 Goal
Extract "discount calculation logic" from src/pages/api/checkout.ts into a standalone service src/services/discountService.ts.

# 🛑 Constraints
- External API Request / Response JSON schemas must maintain strict backward compatibility.
- Strictly avoid impacting other core logic branches inside checkout.ts.

# 🧪 Validation Specs
- Run npm run test to ensure the previously written checkout snapshot tests pass 100% green.
- Run npm run lint with zero type or syntax errors.
```

2. Local Validation and Safe Rollback

Ask Codex to run checks after each small edit. If a test fails, inspect the diff and confirm that there are no human changes to keep before restoring the file:

```
git diff -- src/pages/api/checkout.ts
git restore -- src/pages/api/checkout.ts
```

🛡️ Troubleshooting & Pitfall Cheat Sheet

Common Pitfall	Root Cause	Rapid Diagnosis & Fix Guide
Commanding AI to "refactor the entire project" causing total system collapse	Scope is too broad; AI suffers hallucinations and broken typings across large edits	Strictly constrain refactoring granularity; each prompt must touch only 1 specific service or function
Legacy functionality mysteriously lost or broken after refactoring	Missing pre-refactoring behavioral baseline snapshot tests	Enforce "test-first": require AI to write 100% passing baseline tests against old code before modifying logic
AI arbitrarily upgrades underlying framework packages in package.json	package.json was not locked down in constraints	Add hard rule to AGENTS.md & Constraints: "*Strictly forbid modifying any dependency versions in package.json*"

Ch.10 Validate a Subscription SaaS Payment Flow

> ***Problem**: A subscription product must keep sign-in, payment callbacks, and access state consistent.*
>
> ***Practice**: Use the companion Next.js example to test Stripe payments and webhook signature verification.*

A useful first engineering milestone for a subscription product is a test flow covering sign-in, payment callbacks, and access changes. Real payments also require deployment, risk, and compliance checks.

This chapter uses the companion `Next.js 15 (App Router) + Supabase (PostgreSQL) + Stripe` project to test a local subscription flow. Time required depends on your accounts, environment, and existing code.

> 📦 ***Companion Source Code**: [examples/ch10-saas-mvp] — a fully runnable subscription-based AI translator (TransFlow) with its own CAP `AGENTS.md`. Verified with `npm install && npm run build`.*

A Way to Think About It: Launching a Cash-Generating Street Food Cart

Too many founders try to build a 5-star luxury hotel on Day 1:

```
[Daydreaming 5-Star Hotel] ──> Spending 6 months designing an opulent lobby, importing plush
carpets,
                               hiring dozens of waiters (over-engineering),
                               only to open doors and discover nobody wants to eat there.
Bankruptcy.
[Mobile Street Food Cart]  ──> ✅ You only need 3 essential tools:
                               1. The Griddle (Next.js core product page: delivers immediate
customer value);
                               2. The Storage Box (Supabase / PostgreSQL: stores user accounts
and orders);
                               3. The QR Payment Code (Stripe subscription: collects real money
into your bank).
```

Once a customer scans the code, pays $10, and receives a hot meal in their hands, your commercial loop is validated!

Practice: Start with Three Steps

Set up and verify your local Stripe payment loop in 3 simple steps:

1. **Step 1: Obtain Stripe Test Keys and Declare in `.env.local`**
   Log into the Stripe Developer Dashboard, copy your test keys, and write them into `.env.local`:

   ```
   STRIPE_SECRET_KEY="sk_test_..."
   NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY="pk_test_..."
   ```

2. **Step 2: Start Stripe CLI Local Webhook Forwarding**
   Run the forwarding listener in your terminal to open a local tunnel:

   ```
   stripe listen --forward-to localhost:3000/api/webhooks/stripe
   # The terminal will print your signing secret: whsec_...
   ```

3. **Step 3: Direct Codex in Sandbox to Validate Webhook Signatures**
   Inject `whsec_...` into the environment and instruct Codex: "*Write automated test cases mocking `checkout.session.completed` ; it must return 200 OK and update user subscription status to ACTIVE.*"

10.1 Initialization and Database Schema Design

Our goal is to build a subscription-based AI translation service. First, initialize the project with Next.js 15 and configure core constraints. Next, instruct Codex to generate the database models.

1. Designing the Prisma Schema (Database Entity Modeling)

Dispatch the following goal-driven specs to Codex:

```
# 🎯 Goal
Write Prisma database models supporting User, Subscription, and TranslationRecord entities.

# 🛑 Constraints
- Use PostgreSQL (connected via Supabase) as the database provider.
- Subscription status must be an Enum type containing ACTIVE, CANCELED, and EXPIRED.

# 🧪 Validation Specs
- Running `npx prisma validate` must return no syntax or definition errors.
```

Codex will automatically output a standard `prisma/schema.prisma` file containing foreign key relationships, cascade deletes, and database indexes:

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}

enum SubscriptionStatus {
  ACTIVE
  CANCELED
  EXPIRED
}

model User {
  id             String               @id @default(uuid())
  email          String               @unique
  createdAt      DateTime             @default(now())
  subscription   Subscription?
  records        TranslationRecord[]
}

model Subscription {
  id             String               @id @default(uuid())
  userId         String               @unique
  stripeSubId    String               @unique
  status         SubscriptionStatus
  priceId        String
```

```
  currentPeriodEnd DateTime
  user           User                @relation(fields: [userId], references: [id], onDelete:
Cascade)
}

model TranslationRecord {
  id             String              @id @default(uuid())
  userId         String
  sourceText     String
  translatedText String
  createdAt      DateTime            @default(now())
  user           User                @relation(fields: [userId], references: [id], onDelete:
Cascade)
}
```

10.2 Integrating Stripe Subscriptions and Webhook Handlers

The core of a payment system is **callback security**. When a user successfully checks out, Stripe's servers send a webhook request to your Next.js application.

Practice: Dispatching Webhook Route Specifications to Codex

```
# 🎯 Goal
Implement a Next.js 15 App Router style Stripe Webhook route handler `/api/webhooks/stripe`.

# 🛑 Constraints
- Verify the signature of incoming webhook requests using `stripe.webhooks.constructEvent` to
prevent forgery attacks.
- Update user subscription status in the database on receiving `checkout.session.completed` or
`invoice.payment_succeeded` events.
- Strictly avoid JSON parsing on the request body; Stripe verification requires the raw
`req.text()` string.

# 🧪 Validation Specs
- Write unit tests verifying that invalid signatures return 400, while valid signatures return
200.
- The actual subscription billing period must be retrieved from the Stripe response, never
hardcoded.
```

Codex generates the compliant handler:

```typescript
// File: src/app/api/webhooks/stripe/route.ts
import { NextResponse } from 'next/server';
import Stripe from 'stripe';
import { prisma } from '@/lib/prisma';

const stripe = new Stripe(process.env.STRIPE_SECRET_KEY!, {
  apiVersion: '2026-04-22.dahlia' as any,
});

export async function POST(req: Request) {
  const body = await req.text();
  const signature = req.headers.get('stripe-signature')!;
```

```
  let event: Stripe.Event;

  try {
    event = stripe.webhooks.constructEvent(
      body,
      signature,
      process.env.STRIPE_WEBHOOK_SECRET!
    );
  } catch (err: any) {
    return NextResponse.json({ error: `Webhook Error: ${err.message}` }, { status: 400 });
  }

  if (event.type === 'checkout.session.completed') {
    const session = event.data.object as Stripe.Checkout.Session;
    const stripeSubId = session.subscription as string;
    const customerEmail = session.customer_details?.email!;

    const subscription = await stripe.subscriptions.retrieve(stripeSubId);

    await prisma.subscription.upsert({
      where: { stripeSubId },
      update: {
        status: 'ACTIVE',
        currentPeriodEnd: new Date(subscription.current_period_end * 1000),
      },
      create: {
        stripeSubId,
        status: 'ACTIVE',
        priceId: subscription.items.data[0]?.price.id || 'default',
        currentPeriodEnd: new Date(subscription.current_period_end * 1000),
        user: { connect: { email: customerEmail } }
      }
    });
  }

  return NextResponse.json({ received: true });
}
```

10.3 Local Debugging: Using Stripe CLI for Payment Integration

1. Run Stripe forwarding locally:

```
stripe listen --forward-to localhost:3000/api/webhooks/stripe
```

2. Trigger a real checkout event:

```
stripe trigger checkout.session.completed
```

3. Observe terminal output for `200 OK`, and inspect the database via Prisma Studio:

```
npx prisma studio
```

The value of a commercial MVP lies in delivery speed. Enforce strict boundary specs on the AI to achieve the fastest time-to-revenue.

🛡️ Troubleshooting & Pitfall Cheat Sheet

Common Pitfall	Root Cause	Rapid Diagnosis & Fix Guide
`Webhook Error: No signatures found matching the expected signature`	Used `await req.json()` to parse body, corrupting raw payload bytes	Use `await req.text()` to get the raw string before passing to `constructEvent`
Credit card payment succeeds but database has zero new records	Webhook URL misconfigured or `stripe listen` forwarder not running locally	Run `stripe listen`, confirming listener terminal prints `200 OK [POST /api/webhooks/stripe]`
`DATABASE_URL` cannot connect to Supabase	Forgotten connection pooler (confusing Session vs. Transaction ports)	Verify DB connection string; ensure port 6543 (Transaction mode) is used in Serverless environments

---

Ch.11 Build and Check a Mobile App with Expo

> ***Problem**: Mobile dependencies, device testing, and signed builds can differ by environment.*
>
> ***Practice**: Run the Expo example, check the UI on a device, and configure EAS builds for each platform.*

After shipping a web SaaS, many independent developers want to extend their reach to mobile. However, in traditional mobile development (React Native or Flutter), the most grueling friction is local environment setup: iOS certificates, Android Gradle build failures, and CocoaPods version conflicts.

EAS Build reduces local build setup, but iOS signing, developer accounts, and store review still require configuration. Test on a device before creating development and production builds. Expo build setup

> 📦 ***Companion Source Code**: examples/ch11-expo-mobile — a fully runnable Expo SDK 57 project (Expo Router `src/app` routing + NativeWind + three-tier EAS build profiles) with its own CAP `AGENTS.md`. Verified with `npx expo lint` (zero errors) and `npx expo-doctor` (20/20 checks passed).*

A Way to Think About It: Writing the Script and Letting the "Cloud Atelier" Tailor the Costumes

Cross-platform development shouldn't require turning your personal laptop into a heavy manufacturing plant:

```
[Traditional Native Dev] ──> Buying your own smelting furnace (installing 50GB+ Xcode and
Android Studio),
                             configuring loom machinery (fiddling with Gradle and CocoaPods
runtimes),
                             frequently tripping circuit breakers and stalling for days without
a working build.
[Expo + EAS Mode]        ──> ✅ You only focus on the script (writing React Native / TypeScript
code):
                             - Once the script is ready, send it to the "Cloud Custom Atelier"
(EAS Build);
                             - The cloud automatically tailors iOS IPAs and Android APKs;
                             - Scan the QR code on your phone to slip into the customized outfit
instantly!
```

Codex serves as your script editor in the atelier, automatically aligning versions whenever you miss a dependency or styling quirk.

Practice: Start with Three Steps

See your cross-platform app running on a physical phone in 3 simple steps:

1. **Step 1: Install Expo Go on Your Mobile Device**
   Search for and install "Expo Go" from the App Store or Google Play.
2. **Step 2: Start the Local Development Server**
   Navigate to the project directory and run:

   ```
   npx expo start
   # The terminal will display a large QR code matrix
   ```

3. **Step 3: Scan the QR Code with Your Phone Camera**
   Scan with an Expo Go or development build compatible with this SDK. JavaScript changes usually refresh; native module changes may require a rebuild.

11.1 Expo Project Rapid Initialization and Standards Setup

To use EAS for zero-configuration cloud packaging, initialize a standard Expo project:

1. Writing App Initialization Specs

Issue the following specs to Codex:

```
# 🎯 Goal
Initialize a React Native Expo project using TypeScript.

# 🛑 Constraints
- Use the latest stable Expo SDK 57, paired with Expo Router for file-system-based routing.
- Integrate NativeWind as the Tailwind-style CSS styling solution.
- Use the src/ prefix convention (i.e., src/app/ as the routing root).

# 🧪 Validation Specs
- Running `npx expo lint` must return zero errors.
- Run `npx expo-doctor` to verify dependency version alignment (must pass 20/20).
```

Directory structure layout:

```
src/
├── app/
│   ├── index.tsx         # App Home Page
│   └── _layout.tsx       # Global Routing Navigation Layout
├── components/
└── hooks/
app.json                  # Expo Core Configuration File
package.json
```

11.2 Resolving Mobile Obstacles: Native Module Conflicts

React Native development must avoid using standard `npm install` for third-party packages containing underlying native code (such as cameras, sensors, or gesture handlers).

Golden Rule: Use `npx expo install` for Version Alignment

If dependency versions drift, type this correction directly in the TUI:

```
Please use `npx expo install react-native-reanimated` to reinstall this dependency instead. It will automatically adapt to the current Expo SDK version. In this project, using standard `npm install` to install any native package is strictly prohibited. Please append this rule to AGENTS.md.
```

> 💡 ***Founder's Mantra***: *`npx expo install` selects versions compatible with the current Expo SDK. If a native module fails, run `npx expo-doctor` and inspect dependencies, build logs, and platform limits.*

11.3 EAS Build Cloud Packaging and Certificate Automation

In traditional app delivery, obtaining Apple developer certificates can stall newcomers for days. With EAS, this entire process is handled in the cloud.

1. Configuring `eas.json`

```
{
  "cli": {
    "version": ">= 9.0.0"
{{ ... }}
  }
}
```

2. Instructing Codex to Monitor Cloud Build Logs

```
# Start EAS iOS build task and stream logs to a local file
eas build --platform ios --profile production --non-interactive 2>&1 | tee eas-build.log
```

If the build hits an error, hand the logs directly to Codex for analysis:

```
codex exec --sandbox read-only "Analyze eas-build.log, locate failure cause, and provide remediation commands"
```

Ultimately, EAS returns an installation QR code that you can scan to install the preview app directly on your phone.

🛡️ Troubleshooting & Pitfall Cheat Sheet

Common Pitfall	Root Cause	Rapid Diagnosis & Fix Guide
Phone scans QR code and shows "Could not connect to development server"	Phone and computer are not on the same Wi-Fi LAN, or router firewall blocks local traffic	Run `npx expo start --tunnel` to force public tunnel mode with zero network barriers
`Invariant Violation: "main" has not been registered`	Entry routing file missing or incorrect entry path in `app.json`	Verify `"main": "expo-router/entry"` in `package.json` was not accidentally modified

Terminal throws red screen error immediately after installing a new package	Accidental use of `npm install` introducing an incompatible SDK version	Run `npx expo install <package-name>`, or execute `npx expo-doctor` to diagnose and align

---

Ch.12 From Product Usage to Acquisition Experiments

> ***Problem**: Shipping features does not show whether users complete key flows or where they drop off.*
>
> ***Practice**: Define a few business metrics, build a digest, and test acquisition channels in small reversible experiments.*

On my WeChat public account "Real-World Product Talk" and pmer.cn, I have written numerous articles about "independent development and side hustles." The most common trap developers fall into is: **optimizing implementation details without checking whether real users need the feature.**

No matter how elegant your code is or how perfect your architecture config is, as long as nobody uses it, it is just a pretty ornament. In the AI-native era, we must not only let Codex help us "manufacture products," but also let it help us "spin the commercial flywheel."

A Way to Think About It: Building a Day-and-Night Self-Spinning "Hydraulic Irrigation Waterwheel"

A commercial flywheel shouldn't depend on hauling water buckets by hand every day:

```
[Manual Bucket Hauling] ──> Writing code until midnight, then handing out flyers manually the
next morning.
                            If you get sick for a single day, product updates stall and traffic
drops to zero (fragile).
[Hydraulic Waterwheel]  ──> ✅ You assemble this self-spinning waterwheel by the river:
                            1. The Flume (SEO & automated sitemaps: continuously absorbing long-
tail search traffic);
                            2. The Milling Blades (Core SaaS features: solving real problems for
real users);
                            3. The Grain Funnel (Stripe checkout & subscriptions: continuously
generating cash flow);
                            4. The Water Level Gauge (Daily telemetry cards: monitoring
conversion rates and active users).
```

Once this pipeline is assembled, whether you are sleeping, eating, or traveling, the waterwheel spins day and night, generating compounding value.

Practice: Start with Three Steps

Take your first step toward commercialization in 3 simple steps:

1. **Step 1: Deploy an Automated Sitemap Generation Script**
   Run `scripts/generate-sitemap.js` automatically in your deployment pipeline to ensure every new markdown post or route gets indexed by Google.
2. **Step 2: Hook Up an 8:00 AM Daily Revenue Telemetry Digest**
   Configure a scheduled script that pushes yesterday's new signups and active subscriptions to your phone

every morning, sharpening your business sensitivity.

3. **Step 3: Post Your "One-Sentence Value Proposition" in Public Communities**
   Share your MVP link on X (Twitter), Reddit, or developer forums to gather real payment signals from your first seed cohort.

12.1 Automated Traffic Pipeline for a "One-Person SaaS"

For a healthy indie project, traffic acquisition (SEO, long-tail keywords, social media) should function as an automated conveyor belt just like its codebase.

Practice: Directing Codex to Autonomously Maintain SEO Blogs and Sitemaps

```
// File: scripts/generate-sitemap.js
const fs = require('fs');
const path = require('path');

const BASE_URL = 'https://pmer.cn';
const blogDir = path.join(__dirname, '../content/blog');

function getBlogSlugs() {
  if (!fs.existsSync(blogDir)) return [];
  return fs.readdirSync(blogDir)
    .filter(file => file.endsWith('.md'))
    .map(file => `/blog/${file.replace('.md', '')}`);
}

function generate() {
  const staticPages = ['/', '/auth/login', '/features'];
  const blogPages = getBlogSlugs();
  const allUrls = [...staticPages, ...blogPages];

  const xml = `<?xml version="1.0" encoding="UTF-8"?>
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
  ${allUrls.map(url => `
  <url>
    <loc>${BASE_URL}${url}</loc>
    <changefreq>daily</changefreq>
    <priority>${url === '/' ? '1.0' : '0.8'}</priority>
  </url>`).join('')}
</urlset>`;

  fs.writeFileSync(path.join(__dirname, '../public/sitemap.xml'), xml);
  console.log('✅ Sitemap.xml generated successfully!');
}

generate();
```

12.2 Connecting Business Data for Daily Telemetry

To keep yourself sensitive to cash flow dynamics, fetch Stripe earnings and user growth every morning and broadcast a report straight to your phone:

```
// File: scripts/daily-report.js
const { PrismaClient } = require('@prisma/client');
```

```
const prisma = new PrismaClient();
const axios = require('axios');

async function sendReport() {
  const yesterday = new Date(Date.now() - 24 * 60 * 60 * 1000);
  const userCount = await prisma.user.count({
    where: { createdAt: { gte: yesterday } }
  });

  const activeSubs = await prisma.subscription.count({
    where: { status: 'ACTIVE' }
  });

  const reportText = `📊 [Real-World Product Briefing]\nNew registered users yesterday:
${userCount}\nTotal active subscriptions: ${activeSubs}\n—— Keep going!`;

  await axios.post(process.env.MOBILE_WEBHOOK_URL, {
    msg_type: 'text',
    content: { text: reportText }
  });
}

sendReport();
```

12.3 What Still Matters When Code Is Easier to Produce

As code generation becomes faster, understanding needs, reaching users, and delivering reliable service still determine whether people keep using a product. For independent developers and product managers, useful strengths include:

1. **Understanding the user's actual problem**.
2. **Knowledge of the domain and its constraints**.
3. **The ability to test a product and business assumption with evidence**.

🛡️ Troubleshooting & Pitfall Cheat Sheet

Common Pitfall	Root Cause	Rapid Diagnosis & Fix Guide
Product launched with zero traffic for a month	Siloed development without early SEO or social seeding	Share your building journey on X/Twitter/Indie Hackers immediately; use scripts to output multilingual long-tail blog posts
Obsessing over refactoring and afraid to ship or tweet	Perfectionist trap and fear of public criticism	Remember: as long as core checkout works, ship minor flaws and iterate based on real feedback
Wasting half a day on minor layout tweaks	Failing to delegate visual micro-adjustments to Codex	Use Computer Use (from Ch.07) for automated visual verification, freeing your time to talk to paying customers

---

Ch.13 Check 2026 Model and Tool Changes

> ***Problem***: *Models, clients, and commands change frequently, making older instructions stale.*

> ***Practice**: Check official changes as of this edition and run compatibility checks before migrating.*

The first twelve chapters built a version-independent orchestration methodology. But methodology must land on real tooling. This edition checks changes since September 24: model selection, CLI settings, desktop collaboration, and usage controls. Each migration step identifies its account or client boundary.

A Way to Think About It: Avionics Retrofit of a Supersonic Airliner

Do not panic over frequent tool upgrades. View this transition through the eyes of a pilot:

```
[Pilot Mindset]   ──> Your flight manual remains identical: goal-driven execution (takeoff &
landing targets)
                      and strict boundary constraints (flight corridors & safe altitudes).
[Engines]         ──> Propulsion overhauled to next-generation powerplants:
                      - Frontier Flagship Core: "GPT-6 Astra" (beyond-visual-range radar &
autonomous computer operator);
                      - Daily Primary Thrust Engine: "GPT-6.1 Sol" (complex coding);
                      - Auxiliary Cruise Engine: "GPT-6 Luna" (focused, frequent work).
[Cockpit Screen]  ──> Worktree terminals, review navigation, and mobile approvals improve
collaboration.
                      Check feature access in the installed client and workspace.
[Autopilot]       ──> The old --full-auto compatibility path is deprecated; new calls specify
sandbox and approval modes.
```

With your mental model intact, upgraded avionics only help you fly faster and steadier.

Practice: Start with Three Steps

Complete your 2026 toolchain health check and migration in 3 simple steps:

1. **Step 1: Scan for Deprecated Legacy Model Codenames**
   Run a grep scan in your project root:

   ```
   grep -rn "gpt-5.4" ~/.codex/config.toml .
   ```

2. **Step 2: Check the Installed CLI and Update If Needed** Check the installed version first; run the global npm update when needed:

   ```
   codex --version
   npm install -g @openai/codex@latest
   codex --version
   ```

3. **Step 3: Migrate Deprecated `--full-auto` Calls** In all your shell aliases and CI scripts, replace `--full-auto` with:

   ```
   codex exec --sandbox workspace-write "Review the current changes and report issues"
   ```

13.1 Desktop and Mobile: Changes That Affect Daily Work

Use the Codex entry in your current desktop client and check account and workspace access. The previous blanket claim that a July merger guaranteed every feature to every account has been removed; client packaging and model access need separate checks.

- **Worktrees and steering**: app 26.928 (September 29) improves terminal startup in attached worktrees and keeps earlier answers visible during steering. Later releases improve multi-select chat moves, review navigation, and Windows/WSL recovery. Official changelog
- **Composer predictions**: the October 9 beta suggests a next message in eligible local and SSH threads. Tab accepts an editable draft; sending it is a separate action. Personal Pro, age 18+, supported regions, Astra or GPT-6.1 Sol are required. Toggle it in Settings → General → Composer. Predictions
- **Mobile handoff**: recent iOS releases improve approval presentation, plugin access from the plus menu, and queued-task recovery. Confirm the selected computer and task before approving an operation. Mobile changelog

```
# Check the repository before attaching or handing off a thread.
git status --short
git worktree list
codex --version
```

Check supported sign-in methods and feature access for each client; an API key and a ChatGPT subscription have separate access and billing rules. Codex models

---

13.2 The GPT-6 Family: Choose by Task

As of **October 21, 2026**, GPT-6.1 Sol, launched September 29, is the recommended starting point for complex Codex work when available. Astra remains the choice for the hardest tasks, and Luna for focused work. GPT-6 Sol and GPT-5.6 Sol/Terra/Luna remain available during the rollout. GPT-6.1 Sol launches on Plus, Pro, Business, Enterprise and Edu; administrators must enable it for Enterprise/Edu. Free and Go are excluded at launch. Codex models

Model ID	Best fit	`reasoning.effort`	Boundary
`gpt-6-astra`	The hardest workflows across code, browsers, and desktop tools	`low`, `medium`, `high`, `xhigh`, `max`	No `none`; the client and tools still execute computer actions
`gpt-6.1-sol`	Repeated complex coding and long tasks across apps and documents	`low`, `medium`, `high`, `xhigh`, `max`	Current example default; no `none` or `minimal`
`gpt-6-sol`	Everyday complex coding, code review, and multi-step agent work	`none`, `low`, `medium`, `high`, `xhigh`, `max`	Previous Sol generation; retained for migration comparison
`gpt-6-luna`	Focused edits, extraction, summaries, and high-volume tasks	`none`, `low`, `medium`, `high`, `xhigh`, `max`	Move to Sol or Astra when the task needs more capability

These models accept text and image input and produce text. The API model catalog lists a **1,050,000-token context window** and **128,000 maximum output tokens**. Prefer the Responses API for tools; Astra requires Responses for tool calling. API model catalog · GPT-6 migration guidance

GPT-6.1 Sol migration: the API defaults to `medium`; use Responses for tool calling. Chat Completions supports this model without tool calling. Codex UI efforts can include Max/Ultra depending on settings: Ultra coordinates subagents and is not an API `reasoning.effort` value. API model · Codex effort controls

Keep API prices separate from Codex usage. These are standard API text-token prices per million tokens for requests with up to 272K input tokens, as checked for this edition. They are neither ChatGPT subscription prices nor Codex credit conversions. Longer prompts, caching, and other processing tiers have separate official prices.

API model	Input	Cached input	Output

`gpt-6-astra`	$10	$1	$50
`gpt-6.1-sol`	$2	$0.10	$10
`gpt-6-sol`	$2	$0.20	$10
`gpt-6-luna`	$0.10	$0.01	$0.50

GPT-6.1 Sol cache writes cost $2.50 per million tokens at Standard rates for inputs up to 272K. Longer prompts and speed tiers change these prices. Model pricing

Speed is a separate control. Since October 8, GPT-6.1 Sol Ultrafast is available on Pro $500 and eligible Enterprise/Edu plans, with Enterprise access off until enabled. Included subscription usage consumes 8× Standard limits; purchased credits and Enterprise pay-as-you-go consume 6× Standard. API-key billing uses API prices instead. Sol Ultrafast supports US and EEA + Switzerland inference residency. Start with Standard and compare a familiar task before enabling it. Speed and billing

Choose a model:

```
codex --version
codex --model gpt-6.1-sol
codex exec -m gpt-6-luna "Review the current changes"
```

Use `/model` in an interactive session. If a new model is missing from the picker, check client and account availability. API availability does not imply Codex sign-in availability. Codex models

Local default configuration:

```
# ~/.codex/config.toml
model = "gpt-6.1-sol"
```

This only selects the local default model. The `--approve-for-me` permission mode is not documented as using Luna.

Verified lifecycle dates

Date	Scope	Official record
March 26, 2026	API	`gpt-4-0314`, `gpt-4-1106-preview`, and `gpt-4-0125-preview` shut down
May 12, 2026	API	`dall-e-2` and `dall-e-3` shut down
August 26, 2026	API	Assistants API shut down; migrate to Responses API and Conversations API
August 31, 2026	Codex with ChatGPT sign-in	`gpt-5.4` and `gpt-5.4-mini` retired
September 3, 2026	API	GPT-6 Astra released
September 22, 2026	API / Codex	GPT-6 Sol and Luna released and began rolling out in Codex
September 29, 2026	API / Codex / Work	GPT-6.1 Sol launch

October 14, 2026	Codex with ChatGPT sign-in	GPT-5.5 retired; API and API-key sessions unaffected

Check API deprecations for API shutdown dates and official replacements. Check Codex models and the changelog for Codex sign-in scope. A retirement in one interface does not mean the API model is gone.

13.3 Codex CLI: 0.163.0 at This Edition's Cutoff

The official changelog lists **CLI 0.163.0, released October 15**, at this edition's **October 21, 2026** cutoff. This is a dated verification, not a permanent latest-version claim. Your installed version can differ. Changelog

The 0.159.1 catalog added GPT-6.1 Sol. In 0.163.0, `/config` searches settings and shows restart requirements, `/keymap` configures a leader key, and `/daemon` manages background-server restarts. Steering responds immediately, and resumed threads restore saved permissions subject to current policy. Review potential interruption before restarting a server.

```
codex --version
npm install -g @openai/codex@latest
codex --version
```

1. Migrate older `--full-auto` calls

`codex exec --full-auto` is a **deprecated compatibility path** that prints a warning. It has not been removed. For new non-interactive commands, specify the sandbox:

```
codex exec --sandbox workspace-write "Review the current changes and report issues"
```

The sandbox constrains workspace access. `--approve-for-me` routes eligible escalation requests to automatic review; it does not define the sandbox. Approval and security docs

2. Use the supported Hooks event structure

Hooks in `config.toml` use event arrays and handler arrays. `[hooks] stop = "..."` is not a valid replacement. This example checks whitespace errors in the current repository at the end of a turn. Review and trust the hook source through `/hooks` when first enabled. Hooks docs

```
# ~/.codex/config.toml
[[hooks.Stop]]

[[hooks.Stop.hooks]]
type = "command"
command = "git diff --check"
```

3. Install plugins by IDs shown in the current catalog

```
codex plugin list
codex plugin add --help
```

`codex plugin add` accepts the `PLUGIN@MARKETPLACE` form shown by the catalog. `codex plugin install` is not a current CLI subcommand. Check the ID and source before installing. Codex changelog

4. Automatic approval does not have a documented fixed model

`--approve-for-me` sends eligible approval requests to automatic review. It does not expand sandbox permissions or guarantee approval of every risky action. The documentation does not say the reviewer always runs GPT-6 Luna. [Permissions docs]

```
codex --approve-for-me "Review the current changes and suggest fixes"
```

13.4 The Skills Economy: Compounding Repeated Workflows into Assets

If AGENTS.md is a project's "constitutional law," **Skills are the standard operating procedures for specific task types**:

```
skills/
└── pr-review/
    └── SKILL.md      # A procedural manual with YAML frontmatter
```

13.5 Codex Security: Use Verified Capabilities

Codex Security is a separate security review capability. Check the **Codex Security** entry and instructions in your client's plugin directory before selecting a scan workflow. Commands can differ by version. This book does not present unverified Daybreak model routing, cybersecurity ratings, or scanner flags as executable facts. [Codex Security docs]

```
codex plugin list
```

13.6 A 3D Workflow Example: GPT-6 Astra, Tripo3D, and Blender

This is a workflow developers can assemble themselves: use Astra to structure asset requirements, a 3D service such as Tripo3D to generate a GLB, and a Blender `bpy` script to inspect a render. These are separate tools. **This example does not imply an official OpenAI integration with a third party or guarantee quality or completion time.** The official GPT-6 sources consulted for this edition do not provide the BenchCAD figure published in the previous edition. [Official GPT-6 guide]

1. Workflow boundaries

The model structures requirements and helps write scripts; the 3D service generates a mesh; Blender imports and renders it. A developer reviews geometry, materials, and performance before shipping.

```
Concept and scale → GPT-6 Astra structures prompt → Tripo3D generates GLB
                                                          ↓
                                  Blender bpy render and review → Three.js display
```

2. Four-step walkthrough

Step 1: Structure 3D Prompts via GPT-6 Astra

Instruct Astra in your terminal or desktop prompt:

```
Act as a Lead 3D Technical Artist. I want to build a "Cyberpunk Holographic Vending Machine" for
a Web 3D product showcase.
Generate an engine-optimized, structured 3D prompt for Tripo3D.
```

```
Requirements:
1. Emphasize silhouette and geometric symmetry;
2. Enforce mesh topology tags (Clean quad topology, game-ready, low-poly);
3. Define PBR material specifications (Matte painted metal, glowing neon emissive panels).
```

For example, the model can draft the following prompt for a 3D service; inspect the actual generated result:

```
Cyberpunk holographic vending machine, freestanding rectangular kiosk with chamfered edges.
Center features transparent acrylic dispenser bay with internal glowing LED strip.
Materials: Brushed matte dark gray titanium alloy (roughness: 0.35, metallic: 0.85),
glowing cyan neon emissive trim lines (emission strength: 5.0), scratched hazard decals on base.
Topology tags: clean quad-dominant topology, watertight mesh, manifold geometry, PBR 4K
textures, game-ready asset.
```

Step 2: Generate & Export Standardized GLB via Tripo3D

Enter the prompt in a service such as Tripo3D, inspect the generated mesh and materials, then export `vending_machine.glb` into `public/models/`. Speed and topology quality depend on the service and input.

Step 3: Automate Blender via GPT-6 Astra (`pipeline.py`)

Instead of wrestling with manual GUI dials, have Astra write and execute a headless Blender Python automation script:

```python
import bpy
import math
from pathlib import Path

# 1. Clean factory environment
bpy.ops.wm.read_factory_settings(use_empty=True)

# 2. Import Tripo3D GLB asset
asset_path = Path("public/models/vending_machine.glb").resolve()
if not asset_path.is_file():
    raise FileNotFoundError(asset_path)
bpy.ops.import_scene.gltf(filepath=str(asset_path))

# 3. Compute bounding box, center origin, and ground asset (Z=0)
imported_objs = [obj for obj in bpy.context.selected_objects if obj.type == 'MESH']
if imported_objs:
    bpy.ops.object.select_all(action='DESELECT')
    for obj in imported_objs:
        obj.select_set(True)
    bpy.context.view_layer.objects.active = imported_objs[0]
    bpy.ops.object.origin_set(type='ORIGIN_GEOMETRY', center='BOUNDS')

    min_z = min([v[2] for obj in imported_objs for v in [obj.matrix_world @ v.co for v in
obj.data.vertices]])
    for obj in imported_objs:
        obj.location.z -= min_z

# 4. Automated Studio Three-Point Lighting
# Key Light
key_light_data = bpy.data.lights.new(name="Key_Light", type='AREA')
key_light_data.energy = 500
```

```
key_light_data.size = 2.0
key_light = bpy.data.objects.new(name="Key_Light", object_data=key_light_data)
key_light.location = (3.0, -3.0, 4.0)
bpy.context.collection.objects.link(key_light)

# Fill Light
fill_light_data = bpy.data.lights.new(name="Fill_Light", type='AREA')
fill_light_data.energy = 200
fill_light_data.size = 3.0
fill_light = bpy.data.objects.new(name="Fill_Light", object_data=fill_light_data)
fill_light.location = (-3.0, -2.0, 2.0)
bpy.context.collection.objects.link(fill_light)

# Rim Light
rim_light_data = bpy.data.lights.new(name="Rim_Light", type='SUN')
rim_light_data.energy = 3.0
rim_light = bpy.data.objects.new(name="Rim_Light", object_data=rim_light_data)
rim_light.location = (0.0, 4.0, 3.0)
rim_light.rotation_euler = (math.radians(-45), 0, math.radians(180))
bpy.context.collection.objects.link(rim_light)

# 5. Product Showcase Camera
cam_data = bpy.data.cameras.new("Product_Camera")
cam_obj = bpy.data.objects.new("Product_Camera", cam_data)
cam_obj.location = (0, -4.5, 2.0)
cam_obj.rotation_euler = (math.radians(72), 0, 0)
bpy.context.collection.objects.link(cam_obj)
bpy.context.scene.camera = cam_obj

# 6. Eevee Next Headless Inspection Render
bpy.context.scene.render.engine = 'BLENDER_EEVEE_NEXT'
bpy.context.scene.render.resolution_x = 1280
bpy.context.scene.render.resolution_y = 720
render_path = Path("public/renders/validation_preview.png").resolve()
render_path.parent.mkdir(parents=True, exist_ok=True)
bpy.context.scene.render.filepath = str(render_path)

bpy.ops.render.render(write_still=True)
print("✅ Headless Blender validation frame generated: ./public/renders/validation_preview.png")
```

Execute in background mode:

```
blender --background --python pipeline.py
```

After rendering, provide `validation_preview.png` to a Codex session with image input and ask it to inspect visible geometry and lighting issues. Verify normals, UVs, and performance manually in Blender.

Step 4: Instant Web Embed (React Three Fiber / Three.js)

Distribute the optimized GLB asset directly inside your Next.js 15 or Vite web application:

```
// components/VendingMachineViewer.tsx
import { Canvas } from '@react-three/fiber';
import { useGLTF, OrbitControls, Environment } from '@react-three/drei';

export function VendingMachineViewer() {
```

```
  const { scene } = useGLTF('/models/vending_machine.glb');
  return (
    <div className="w-full h-[500px] bg-slate-900 rounded-xl overflow-hidden shadow-2xl">
      <Canvas camera={{ position: [0, 2, 4], fov: 45 }}>
        <ambientLight intensity={0.7} />
        <directionalLight position={[5, 10, 5]} intensity={1.2} />
        <primitive object={scene} position={[0, -1, 0]} />
        <Environment preset="city" />
        <OrbitControls autoRotate autoRotateSpeed={2.0} enableZoom={true} />
      </Canvas>
    </div>
  );
}
```

13.7 Cost and Task-Based Routing

Start everyday complex coding with `gpt-6.1-sol`, use `gpt-6-luna` for focused extraction and small edits, and evaluate `gpt-6-astra` for the hardest cross-app work. Check what your account and client can select, then compare quality, latency, and cost on your own tasks. Codex models

If a project uses other providers, verify session formats and tool support independently. The companion Codex Switch is a third-party tool; rely on its current documentation and your own compatibility tests.

13.8 Migration Checklist

```
codex --version
npm install -g @openai/codex@latest
codex --version
codex --model gpt-6.1-sol
codex exec --sandbox workspace-write "Review the current changes and report issues"
codex plugin list
```

1. Check GPT-6 access for your account and workspace in Codex models.
2. Search saved `gpt-5.4`, `gpt-5.4-mini`, and `--full-auto` settings; migrate each according to its actual scope.
3. If using Hooks, verify the event-array structure in the Hooks docs and inspect trust with `/hooks`.

Troubleshooting

Symptom	Check
GPT-6.1 Sol or Luna is absent from the picker	Check CLI version, plan, rollout, and workspace administrator settings
`--full-auto` prints a deprecation warning	Use `--sandbox workspace-write` for new non-interactive calls; the old path remains compatible
A configured model is unavailable	Distinguish Codex sign-in from API access and select an available model with `/model`
Hooks do not run	Check the event-array structure and review hook trust with `/hooks`

13.9 Principles That Outlast Versions

Keep changing model IDs and CLI options in one place. Check official documentation and local behavior during each upgrade. Continue to state the task goal, permission boundary, and acceptance criteria clearly.

Related tool channel

For concise updates and setup reminders, visit the PMER Codex tool channel. It uses the same October 21, 2026 baseline; use this chapter for configuration details.
